

Notes of DBMS

Overview of Database, Database Management System (DBMS), DBMS Architecture, Data Independence, Integrity Constraints

1. Overview of Database

- **Definition:** A database is an organized collection of structured data that is stored electronically and can be efficiently retrieved, managed, and updated.
- **Purpose:** Databases are used to store and manage data in a way that ensures easy access, retrieval, and management of information.
- **Examples:** A university storing student records, an e-commerce platform storing product catalogs, etc.

2. Database Management System (DBMS)

- **Definition:** A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to the database. It acts as an interface between the users and the database, ensuring efficient data management.
- **Functions:**
 1. **Data Storage:** Manages how data is stored on disk.
 2. **Data Retrieval:** Provides query processing capabilities (e.g., SQL) to retrieve data efficiently.
 3. **Data Manipulation:** Allows for data modification (insert, update, delete).
 4. **Data Security:** Controls access to the database, ensuring that only authorized users can perform specific operations.
 5. **Concurrency Control:** Ensures that multiple users can access the database simultaneously without conflicts.
 6. **Backup and Recovery:** Helps recover the database to a consistent state in case of a failure.
- **Examples of DBMS:** MySQL, Oracle, PostgreSQL, Microsoft SQL Server.

3. DBMS Architecture

DBMS architecture describes the structure and organization of the DBMS components. It is typically divided into the following layers:

a) 1-Tier Architecture:

- **Definition:** In 1-tier architecture, the DBMS and user interface both reside on the same machine. The database directly interacts with the user without any intermediary layers.
- **Example:** A stand-alone database application like Microsoft Access.

b) 2-Tier Architecture:

- **Definition:** In 2-tier architecture, the DBMS is located on a server, and the application (or client) resides on the user's machine. The client directly communicates with the database server.
- **Components:**
 1. **Client Application:** Sends queries and receives responses from the DBMS.
 2. **Database Server:** Manages the actual data storage and processing.
- **Example:** A desktop application connecting to a remote database server.

c) 3-Tier Architecture:

- **Definition:** In 3-tier architecture, an intermediate layer (application server or business logic layer) exists between the client and the database server. It allows for more scalability and flexibility.
- **Components:**
 1. **Presentation Layer:** The client interface that users interact with.
 2. **Application Layer (Business Logic):** Handles the business logic, processing the client's request before sending it to the database.
 3. **Database Layer:** The backend where data is stored and managed.
- **Example:** A web-based application where the user interface is a browser, the application server processes the requests, and the database server stores the data.

4. Data Independence

Data independence refers to the capacity to change the database schema without affecting the higher-level application programs. It is classified into two types:

a) Logical Data Independence:

- **Definition:** The ability to change the logical schema (e.g., adding new tables or fields) without affecting the external schema (user views or application programs).

- **Importance:** This allows flexibility in the logical design and structure of the database without disturbing how users access the data.

b) Physical Data Independence:

- **Definition:** The ability to change the physical storage structure (e.g., indexing, storage devices) without affecting the logical schema or application programs.
- **Importance:** This allows for optimizations at the storage level without disrupting the application-level functionality.

5. Integrity Constraints

Integrity constraints are rules that ensure the accuracy and consistency of data in a database. They are enforced to maintain data quality and avoid anomalies.

a) Types of Integrity Constraints:

1. Domain Constraints:

- **Definition:** Specifies the permissible values that a column in a table can hold. For example, the age field can only hold integer values between 0 and 150.
- **Example:** CHECK (age >= 0 AND age <= 150) ensures that the age is within the valid range.

2. Entity Integrity Constraint:

- **Definition:** Ensures that every table has a **primary key**, and that the primary key cannot be NULL. This guarantees that each record in the table is uniquely identifiable.
- **Example:** A "Student ID" column in a student table must be unique and cannot contain NULL.

3. Referential Integrity Constraint:

- **Definition:** Ensures that relationships between tables remain consistent. It enforces that a **foreign key** in one table matches a valid **primary key** in another table.
- **Example:** In an "Orders" table, the "Customer ID" field must refer to a valid "Customer ID" in the "Customers" table.

4. Key Constraints:

- **Definition:** A rule that requires a set of attributes (columns) in a relation (table) to uniquely identify a record. This is usually enforced using **primary keys** and **unique constraints**. (unique constraint is known as candidate key)
- **Example:** A unique constraint on "Email" in a user table ensures that no two users have the same email address.

5. NOT NULL Constraint:

- **Definition:** Ensures that a column cannot have NULL values. It is applied when a column must have a value for every record.
- **Example:** A "Phone Number" column in a contacts table must not contain any NULL values.

b) Importance of Integrity Constraints:

- **Data Accuracy:** Ensures that only valid and consistent data is entered into the database.
- **Data Consistency:** Maintains the relationship between different tables, ensuring no data anomalies.
- **Prevents Data Redundancy:** Constraints like primary keys and foreign keys help eliminate redundancy and enforce relationships.
- **Maintains Business Rules:** Ensures that the data adheres to specific rules and conditions defined by the business.

Data Models, ER (Entity Relationship) Diagram ([Hierarchical Data Model is the oldest.](#))

1. Data Models

A **data model** defines how data is structured, stored, and manipulated in a database. It provides a conceptual framework for representing real-world entities and their relationships in a structured manner.

a) Types of Data Models:

There are several types of data models, each serving different purposes:

1. Hierarchical Data Model:

- **Structure:** Data is organized in a tree-like structure, where each record has a single parent and can have multiple children.
- **Example:** An organizational chart where each employee reports to one manager but can manage multiple subordinates.
- **Advantages:**
 - Simple and efficient for hierarchical relationships.
- **Disadvantages:**
 - Limited flexibility; difficult to represent many-to-many relationships.

2. Network Data Model:

- **Structure:** Similar to the hierarchical model, but records can have multiple parent and child relationships. It uses a graph-like structure.
- **Example:** A university system where students can enroll in multiple courses, and each course can have multiple students.
- **Advantages:**
 - Supports many-to-many relationships.
- **Disadvantages:**
 - Complex to implement and maintain.

3. Relational Data Model:

- **Structure:** Data is organized in tables (called **relations**), where each table consists of rows and columns. Tables can relate to each other via keys (primary and foreign keys).
- **Example:** A customer database where "Customers" and "Orders" tables are linked via a "CustomerID" foreign key.
- **Advantages:**
 - Flexibility, scalability, and easy querying using SQL.
- **Disadvantages:**
 - Overhead in managing large numbers of relations and maintaining consistency.

4. Object-Oriented Data Model:

- **Structure:** Data is represented as objects, similar to object-oriented programming concepts, where objects encapsulate both data and behavior.
- **Example:** A library system where each "Book" object has properties (title, author) and behaviors (borrow, return).
- **Advantages:**
 - Supports complex data types and object inheritance.
- **Disadvantages:**
 - Can be slower for simple queries compared to relational models.

5. Entity-Relationship Model (ER Model):

- **Structure:** A conceptual representation of data using entities (objects) and relationships between them. The ER model helps design the database at a high level before converting it into a relational schema.

- **Example:** An ER diagram showing entities like "Student", "Course", and relationships like "Enrolls In".
- **Advantages:**
 - High-level view of the database structure before implementation.
- **Disadvantages:**
 - Needs to be converted into a relational or other concrete model for implementation.

2. ER (Entity-Relationship) Diagram

An **Entity-Relationship (ER) diagram** is a graphical representation of entities and their relationships in a database. It is a key component in database design, helping visualize the structure and ensure proper relationships among data elements.

a) Components of an ER Diagram:

1. Entities:

- **Definition:** Entities are objects or things in the real world that are distinguishable and have a set of attributes.
- **Example:** "Student", "Teacher", "Product", etc.
- **Notation:** Represented by rectangles in the ER diagram.
- **Entity Types:** Can be strong or weak entities.
 - **Strong Entity:** An independent entity that can exist without relying on another entity.
 - **Weak Entity:** An entity that depends on another entity for its existence (e.g., "Order Item" depends on "Order").

2. Attributes:

- **Definition:** Attributes are properties or characteristics of an entity.
- **Example:** For a "Student" entity, attributes could be "StudentID", "Name", "Age", etc.
- **Types of Attributes:**
 - **Simple Attribute:** Contains atomic values (e.g., "Age").
 - **Composite Attribute:** Can be divided into smaller sub-parts (e.g., "Name" can be divided into "First Name" and "Last Name").
 - **Multivalued Attribute:** An attribute that can have multiple values (e.g., a "Phone Numbers" attribute may contain more than one number).

- **Derived Attribute:** An attribute whose value is derived from other attributes (e.g., "Age" can be derived from the "Date of Birth").

- **Notation:** Represented by ovals connected to entities.

3. Relationships:

- **Definition:** A relationship describes how two entities are associated with each other.
- **Example:** A "Student" **enrolls** in a "Course", a "Customer" **places** an "Order".
- **Notation:** Represented by diamonds in the ER diagram.
- **Types of Relationships:**
 - **One-to-One (1:1):** One entity is associated with one other entity (e.g., each person has one passport).
 - **One-to-Many (1**

): One entity is associated with many other entities (e.g., a "Teacher" teaches many "Courses").

- **Many-to-Many (M**

): Many entities are associated with many others (e.g., students enroll in many courses, and each course has many students).

4. Cardinality:

- **Definition:** Cardinality defines the number of instances of one entity that can be associated with the number of instances in another entity.
- **Types:**
 - **1:1:** One instance of Entity A is related to one instance of Entity B.
If one teacher teaches only one class, and each class has only one teacher.
 - **1:M**
If one teacher can teach many classes, but each class has only one teacher.

: One instance of Entity A is related to multiple instances of Entity B.

- **M:M**
If one teacher can teach many classes, and each class can have many teachers.

: Multiple instances of Entity A are related to multiple instances of Entity B.

5. Primary Key:

- **Definition:** A primary key is an attribute (or combination of attributes) that uniquely identifies each instance of an entity.
- **Example:** "StudentID" uniquely identifies each student in the "Student" entity.

6. Foreign Key:

- **Definition:** A foreign key is an attribute in one entity that refers to the primary key of another entity to establish a relationship between the two.
- **Example:** "CourseID" in the "Enrollment" entity refers to "CourseID" in the "Course" entity.

7. Weak Entities and Identifying Relationships:

- **Weak Entity:** An entity that cannot exist without a strong (parent) entity.
- **Identifying Relationship:** The relationship between a strong and weak entity.
- **Example:** "Order Item" is a weak entity that cannot exist without an "Order".

b) Steps to Create an ER Diagram:

1. **Identify Entities:** Determine the objects or things that need to be represented (e.g., "Customer", "Product").
2. **Identify Relationships:** Define how the entities are related to one another (e.g., "Customer places Order").
3. **Identify Attributes:** List the properties of each entity (e.g., "Customer Name", "Order Date").
4. **Determine Cardinality:** Specify the type of relationship between entities (1:1, 1, M, M).
5. **Draw the Diagram:** Use the notation (rectangles for entities, diamonds for relationships, ovals for attributes) to visually represent the data model.

c) Example of an ER Diagram:

Let's take a university scenario:

- **Entities:**
 - **Student** (Attributes: StudentID, Name, DOB)
 - **Course** (Attributes: CourseID, CourseName)
 - **Enrollment** (Attributes: EnrollmentID, Grade)
- **Relationships:**
 - **Enrolls In:** A "Student" can enroll in multiple "Courses" (1, M), and a "Course" can have multiple "Students" (M, 1).
 - **Taught By:** Each "Course" is taught by one "Professor", and a "Professor" can teach multiple "Courses" (1, M).

).

Functional Dependencies, Introduction of Normalization and its importance, Data Redundancy and Update Anomalies

1. Functional Dependencies

a) Definition:

A **functional dependency (FD)** is a relationship between two sets of attributes in a relation (table), where one set of attributes determines another. It is a fundamental concept used to identify redundancy in a database.

- **Formal Definition:** Given a relation R, an attribute Y is functionally dependent on an attribute X (denoted as $X \rightarrow Y$) if, for every instance of X, there is exactly one associated instance of Y.

b) Types of Functional Dependencies:

1. Trivial Functional Dependency:

- **Definition:** A functional dependency is trivial if the dependent attribute set is a subset of the determinant attribute set.
- **Example:** If $A \rightarrow A$ or $A, B \rightarrow A$, these are trivial dependencies because the right-hand side is already part of the left-hand side.

2. Non-Trivial Functional Dependency:

- **Definition:** A dependency is non-trivial if the right-hand side is not a subset of the left-hand side.
- **Example:** $A \rightarrow B$ is non-trivial if B is not part of A.

3. Partial Dependency:

- **Definition:** A functional dependency is partial if a non-prime attribute is functionally dependent on part of a candidate key.
- **Example:** In a relation with candidate key A, B, if $A \rightarrow C$, but B is not necessary to determine C, it's a partial dependency.

4. Transitive Dependency:

- **Definition:** A transitive dependency occurs when a non-prime attribute depends on another non-prime attribute through a chain of dependencies.
- **Example:** If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency.

c) Importance of Functional Dependencies:

- Functional dependencies are essential for **normalization** since they help identify redundancy and ensure the correctness of database design.
- They aid in eliminating **anomalies** like insertion, deletion, and update anomalies.

2. Introduction to Normalization and Its Importance

a) Definition of Normalization:

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing large tables into smaller, more manageable tables and defining relationships between them. The goal is to eliminate undesirable characteristics like data anomalies and data duplication.

b) Normal Forms:

There are several levels of normalization, called **normal forms**, each addressing specific types of redundancy and anomalies:

1. First Normal Form (1NF):

- **Requirements:** The table should only contain atomic (indivisible) values. Repeating groups or arrays are not allowed.
- **Example:** A column containing multiple phone numbers should be split into separate rows, each with one phone number.

2. Second Normal Form (2NF):

- **Requirements:** The table must first satisfy 1NF and eliminate **partial dependencies**, meaning every non-prime attribute must depend on the entire primary key, not just a part of it.
- **Example:** If a table has a composite key (e.g., "OrderID, ProductID") and a column (e.g., "ProductName") depends only on "ProductID", then "ProductName" should be moved to a separate table.

3. Third Normal Form (3NF):

- **Requirements:** The table must satisfy 2NF and eliminate **transitive dependencies**, meaning no non-prime attribute should depend on another non-prime attribute.
- **Example:** If "CustomerCity" depends on "CustomerID" and "CustomerCity" also determines "ShippingRegion", then "ShippingRegion" should be placed in a separate table.

4. Boyce-Codd Normal Form (BCNF):

- **Requirements:** A stricter version of 3NF. Every determinant must be a candidate key.
- **Example:** If a non-candidate key column determines part of the primary key, BCNF suggests restructuring the table.

c) Importance of Normalization:

- **Reduces Data Redundancy:** By organizing data into multiple related tables, normalization minimizes the duplication of data across the database.
- **Prevents Anomalies:** Normalization prevents **update, insertion, and deletion anomalies** (discussed below).
- **Improves Data Integrity:** Ensures that the relationships between tables are consistent and accurate, reducing the risk of inconsistencies.
- **Optimizes Database Structure:** Normalization enhances the structure of the database, making it more efficient in terms of storage and retrieval.

3. Data Redundancy

a) Definition:

Data redundancy occurs when the same piece of data is stored in multiple places within a database. This duplication of data leads to increased storage usage and the potential for data inconsistencies.

b) Problems Caused by Data Redundancy:

1. **Increased Storage Costs:** Storing the same data multiple times unnecessarily increases the storage required.
2. **Data Inconsistency:** Redundant data may lead to inconsistencies when different copies of the same data are updated independently, causing mismatches.
 - **Example:** If a customer's address is stored in multiple tables and updated in only one place, the data becomes inconsistent.
3. **Maintenance Issues:** Managing and synchronizing redundant data requires more effort, leading to complex and error-prone updates.

c) How Normalization Reduces Redundancy:

Normalization reduces data redundancy by breaking larger tables into smaller, related tables, ensuring that each piece of data is stored in only one place and can be referenced through foreign keys when necessary.

4. Update Anomalies

a) Definition:

Update anomalies occur when changes to the data are incorrectly or inefficiently propagated across the database, leading to inconsistencies. These anomalies are often caused by poor database design or data redundancy.

b) Types of Update Anomalies:

1. **Insertion Anomaly:**
 - **Definition:** Inability to insert new data due to the absence of other related data.
 - **Example:** In a student-course relation, if you want to insert a new course that no students are enrolled in yet, but the database design forces you to provide student information, it causes an insertion anomaly.

- **Solution:** Create a separate "Courses" table to store course information independently of student enrollments.

2. Deletion Anomaly:

- **Definition:** Unintentional loss of data when deleting a record.
- **Example:** If deleting a student record from a table that also contains course details results in the loss of course information, this is a deletion anomaly.
- **Solution:** Store course information in a separate table so it isn't affected by deleting student records.

3. Update Anomaly:

- **Definition:** Inconsistent data results when updating one instance of a piece of data while other instances remain unchanged.
- **Example:** If a customer's phone number is stored in multiple tables, updating the phone number in one table but not in others leads to inconsistent data.
- **Solution:** Normalize the data to ensure the phone number is stored in only one table, with references (foreign keys) in other tables.

c) How Normalization Solves Update Anomalies:

- **Insertion Anomaly:** By structuring data into independent tables, normalization ensures that new data can be inserted without needing unrelated data.
- **Deletion Anomaly:** Normalized tables ensure that deleting a record does not unintentionally remove related but independent data.
- **Update Anomaly:** By removing data redundancy, normalization ensures that updates to data are made in only one place, preventing inconsistencies.

Normal Forms (1NF, 2NF, 3NF & BCNF), De-Normalization

1. Normal Forms

Normal forms are standards or guidelines for organizing databases to reduce redundancy and prevent anomalies like insertion, deletion, and update issues. The goal is to structure data efficiently and maintain data integrity. The most commonly used normal forms are **First Normal Form (1NF)**, **Second Normal Form (2NF)**, **Third Normal Form (3NF)**, and **Boyce-Codd Normal Form (BCNF)**.

a) First Normal Form (1NF)

- **Definition:** A relation is in **1NF** if:
 1. All the values in the columns are atomic (indivisible).
 2. There are no repeating groups or arrays.
- **Conditions:**
 1. Each column must contain unique and atomic values.

2. There should be no duplicate rows (must have a unique identifier, i.e., a primary key).

- **Example:** Consider the following table, which violates 1NF due to repeating groups:

StudentID Name Subjects

101	John	Math, Physics
102	Alice	Chemistry

- To convert this to 1NF, we split the subjects into separate rows:

StudentID Name Subject

101	John	Math
101	John	Physics
102	Alice	Chemistry

b) Second Normal Form (2NF)

- **Definition:** A relation is in **2NF** if:
 1. It is in 1NF.
 2. All non-key attributes are fully functionally dependent on the entire primary key (no **partial dependencies**).
- **Explanation:** A **partial dependency** occurs when a non-key attribute is dependent on part of a composite primary key. In 2NF, such partial dependencies are removed.
- **Example:** Consider the following table:

StudentID CourseID CourseName

101	C101	Math
102	C102	Chemistry

- Here, CourseName depends only on CourseID (not the entire primary key StudentID, CourseID), which is a partial dependency. To convert this into 2NF, split the table as follows:
- **Student-Course Table:**

StudentID CourseID

101	C101
102	C102

- **Course Table:**

CourseID CourseName

C101 Math

C102 Chemistry

c) Third Normal Form (3NF)

- **Definition:** A relation is in **3NF** if:
 1. It is in 2NF.
 2. There is no **transitive dependency**.
- **Explanation:** A **transitive dependency** exists if a non-key attribute depends on another non-key attribute rather than directly on the primary key.
- **Example:** Consider the following table:

EmployeeID DepartmentID DepartmentName

201 D01 Sales

202 D02 HR

- Here, DepartmentName depends on DepartmentID, which in turn depends on EmployeeID. This creates a transitive dependency. To bring it into 3NF, split the table as follows:
- **Employee Table:**

EmployeeID DepartmentID

201 D01

202 D02

- **Department Table:**

DepartmentID DepartmentName

D01 Sales

D02 HR

d) Boyce-Codd Normal Form (BCNF)

- **Definition:** A relation is in **BCNF** if:
 1. It is in 3NF.
 2. For every functional dependency $X \rightarrow Y$, X should be a superkey (a key that uniquely identifies rows in the table).
- **Explanation:** BCNF is a stricter version of 3NF. It deals with certain types of anomalies that 3NF cannot handle. BCNF ensures that every determinant (the left side of the functional dependency) is a candidate key.

- **Example:** Consider the following table:

Professor Course Department

Dr. Smith Math Science

Dr. Jones History Humanities

- Here, Professor determines Department, but Course also determines Department. This violates BCNF because Course is not a candidate key, yet it determines Department. To convert this table into BCNF, we split it as follows:

- **Professor Table:**

Professor Course

Dr. Smith Math

Dr. Jones History

- **Course-Department Table:**

Course Department

Math Science

History Humanities

2. De-Normalization

a) Definition:

De-normalization is the process of combining tables that were separated during normalization to improve database performance. It involves intentionally introducing redundancy into a database schema to reduce the time taken to retrieve data by minimizing the number of joins required.

b) When to Use De-Normalization:

De-normalization is typically used in situations where:

- **Performance** is more critical than redundancy, and frequent joins across tables slow down query execution.
- **Data retrieval** needs to be optimized for read-heavy applications, such as reporting systems.

c) Example of De-Normalization:

Consider the normalized structure of Customer, Order, and Product tables:

Customer Table:

CustomerID CustomerName

C01 John

CustomerID CustomerName

C02 Alice

Order Table:

OrderID CustomerID ProductID Quantity

O01 C01 P01 5

O02 C02 P02 3

Product Table:

ProductID ProductName Price

P01 Laptop 500

P02 Phone 300

In a normalized database, retrieving the complete order details would require joining these three tables. To optimize for faster retrieval, we can de-normalize by combining them into a single table:

De-Normalized Table:

OrderID CustomerName ProductName Quantity Price

O01 John Laptop 5 500

O02 Alice Phone 3 300

d) Advantages of De-Normalization:

1. **Improved Performance:** Queries can retrieve data faster since fewer joins are needed.
2. **Simplified Queries:** Fewer joins result in simpler and more readable queries.

e) Disadvantages of De-Normalization:

1. **Increased Redundancy:** Data redundancy is introduced, leading to potential inconsistencies.
2. **Update Anomalies:** With data duplicated across multiple places, updating one piece of data (e.g., product price) requires updating it in multiple locations.
3. **More Storage Required:** Since the same data is repeated, de-normalization increases the amount of storage used.

Summary of Normalization vs. De-Normalization:

- **Normalization:** Focuses on eliminating redundancy and avoiding anomalies by splitting tables and ensuring data integrity.
- **De-Normalization:** Intentionally combines tables to optimize for performance, accepting some redundancy and potential anomalies for faster query execution.

Introduction to SQL, SQL Commands, Datatypes, DDL statements (create, alter, drop and truncate)

Introduction to SQL

a) What is SQL?

- **SQL (Structured Query Language)** is a standard programming language specifically designed for managing and manipulating relational databases.
- SQL is used for querying, updating, and managing data in relational databases like MySQL, PostgreSQL, SQL Server, and Oracle.
- It is a declarative language, meaning users specify **what** data they need without having to specify **how** to retrieve it.

b) Key Features of SQL:

- **Data Retrieval:** SQL allows you to query the database and retrieve data using SELECT statements.
 - **Data Manipulation:** You can add, update, or delete records in a database using commands like INSERT, UPDATE, and DELETE.
 - **Database Management:** SQL lets you create, modify, and manage databases and their structure using DDL commands (CREATE, ALTER, DROP).
 - **Security:** SQL provides features like roles, permissions, and views to control who can access and manipulate the data.
-

SQL Commands

SQL commands are categorized into several types based on their functionality. The main categories are:

a) DML (Data Manipulation Language):

DML commands are used for modifying data stored in a database.

- **SELECT:** Used to retrieve data from a table.
- **INSERT:** Used to add new data to a table.
- **UPDATE:** Used to modify existing data in a table.
- **DELETE:** Used to remove records from a table.

b) DDL (Data Definition Language):

DDL commands deal with the structure or schema of the database and its objects like tables, indexes, and views.

- **CREATE:** Used to create new database objects (tables, views, indexes).
- **ALTER:** Used to modify the structure of existing database objects.
- **DROP:** Used to delete database objects.
- **TRUNCATE:** Used to remove all records from a table without deleting the table itself.

c) DCL (Data Control Language):

DCL commands are used to control access to the data stored in a database.

- **GRANT:** Gives user access privileges to the database.
- **REVOKE:** Removes user access privileges.

d) TCL (Transaction Control Language):

TCL commands are used to control transactions in a database.

- **COMMIT:** Saves the changes made by a transaction.
- **ROLLBACK:** Undoes changes made by a transaction before they are committed.
- **SAVEPOINT:** Sets a point within a transaction to which you can later roll back.

e) DQL (Data Query Language):

- **SELECT:** The only command in this category, used to query the database and retrieve data from one or more tables.

Data Types in SQL

SQL supports various data types, which are used to define the type of data that can be stored in each column of a table. These data types vary slightly between different SQL implementations, but the main categories are:

a) Numeric Data Types:

- **INT or INTEGER:** Used for whole numbers (e.g., INT, BIGINT, SMALLINT).
- **FLOAT, DOUBLE:** Used for real numbers with floating-point precision.
- **DECIMAL(p, s) or NUMERIC(p, s):** Used for fixed-point numbers, with precision p and scale s.

b) Character/String Data Types:

- **CHAR(n):** Fixed-length string, where n defines the length (e.g., CHAR(10)).
- **VARCHAR(n):** Variable-length string, with a maximum length of n (e.g., VARCHAR(50)).
- **TEXT:** Large variable-length string (usually for longer text data).

c) Date and Time Data Types:

- **DATE:** Used to store dates (format: YYYY-MM-DD).
- **TIME:** Used to store time (format: HH:MM:SS).

Precision (p): This defines the total number of digits that the value can have, including both the integer and fractional parts. For example, a DECIMAL(5, 2) can have up to 5 digits in total, with 2 of those digits after the decimal point.

Scale (s): This specifies how many of the digits will appear after the decimal point. So, in DECIMAL(5, 2), the scale is 2, meaning 2 digits will be stored after the decimal point.

Example values: 123.45, 9.99, 100.00, 0.01.

- **DATETIME:** Combines both date and time.
- **TIMESTAMP:** Stores date and time, often used for tracking records creation or modification time.

d) Boolean Data Type:

- **BOOLEAN:** Used to store TRUE or FALSE values.

e) Binary Data Types:

- **BLOB (Binary Large Object):** Used to store binary data such as images, videos, or audio files.

DDL Statements

DDL (Data Definition Language) consists of commands that define and modify the structure of database objects like tables, indexes, and views. The key DDL commands are CREATE, ALTER, DROP, and TRUNCATE.

1. CREATE Statement

The CREATE statement is used to create a new database object, such as a table, view, or index.

- **Syntax for Creating a Table:**

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    ...
);
```

- **Example:**

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    salary DECIMAL(10, 2),
    hire_date DATE
);
```

This creates an employees table with 5 columns: employee_id, first_name, last_name, salary, and hire_date.

2. ALTER Statement

The ALTER statement is used to modify the structure of an existing database object. It can be used to add, modify, or drop columns.

- **Syntax:**

```
ALTER TABLE table_name
```

```
ADD column_name datatype;
```

```
ALTER TABLE table_name
```

```
MODIFY column_name new_datatype;
```

```
ALTER TABLE table_name
```

```
DROP column_name;
```

- **Example (Add a column):**

```
ALTER TABLE employees
```

```
ADD email VARCHAR(100);
```

This adds a new email column to the employees table.

- **Example (Modify a column):**

```
ALTER TABLE employees
```

```
MODIFY salary DECIMAL(12, 2);
```

This changes the salary column to have more precision.

3. DROP Statement

The DROP statement is used to delete an entire database object, such as a table or a view.

- **Syntax:**

```
DROP TABLE table_name;
```

```
DROP VIEW view_name;
```

- **Example:**

DROP TABLE employees;

This deletes the employees table, along with all the data stored in it.

4. TRUNCATE Statement

The TRUNCATE statement is used to remove all records from a table, but it does not delete the table itself. Unlike DELETE, it is faster and cannot be rolled back.

- **Syntax:**

TRUNCATE TABLE table_name;

- **Example:**

TRUNCATE TABLE employees;

This removes all records from the employees table, but the table structure remains.

Difference Between DROP and TRUNCATE:

- DROP removes the entire table, including its structure.
- TRUNCATE removes all data from the table but retains the structure for future use.

Summary

- **SQL** is a powerful language for managing relational databases.
- SQL commands are categorized into DML (for data manipulation), DDL (for defining data structures), DCL (for controlling access), and TCL (for managing transactions).
- **Data types** help define the type of data that can be stored in a table, ensuring data integrity.
- **DDL statements** (CREATE, ALTER, DROP, and TRUNCATE) are used to define and modify database objects like tables.

[DML Statements \(insert, update, delete\), Simple queries WHERE Clause, Compound WHERE Clause with multiple AND & OR Conditions Joins](#)

DML Statements (Data Manipulation Language)

DML statements in SQL are used to manage and manipulate the data stored in the database. The most commonly used DML commands are INSERT, UPDATE, and DELETE.

1. INSERT Statement

The **INSERT** statement is used to add new records to a table. You can insert a single record or multiple records into a table.

a) Basic Syntax for Single Row Insert:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

- **Example:**

```
INSERT INTO employees (employee_id, first_name, last_name, department_id)  
VALUES (101, 'John', 'Doe', 10);
```

This inserts a new row with `employee_id` as 101, `first_name` as 'John', `last_name` as 'Doe', and `department_id` as 10 into the `employees` table.

b) Inserting Multiple Rows:

```
INSERT INTO employees (employee_id, first_name, last_name, department_id)  
VALUES  
(102, 'Jane', 'Smith', 20),  
(103, 'Robert', 'Brown', 30);
```

This inserts two rows into the `employees` table at once.

2. UPDATE Statement

The **UPDATE** statement is used to modify existing data in a table. You can update one or more records based on a specific condition.

a) Basic Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

- **Example:**

```
UPDATE employees  
SET department_id = 15
```

WHERE employee_id = 101;

This updates the department_id for the employee with employee_id 101 to 15.

- **Example (Updating Multiple Columns):**

UPDATE employees

SET first_name = 'John', last_name = 'Smith'

WHERE employee_id = 101;

This updates the first_name and last_name of the employee with employee_id 101.

3. DELETE Statement

The **DELETE** statement is used to remove records from a table. The deletion is permanent unless rolled back using a transaction control command.

a) Basic Syntax:

DELETE FROM table_name

WHERE condition;

- **Example:**

DELETE FROM employees

WHERE employee_id = 103;

This deletes the row where employee_id is 103.

- **Deleting All Rows** (Be cautious):

DELETE FROM employees;

This removes all rows from the employees table, but keeps the table structure intact.

Simple Queries with WHERE Clause

The **WHERE** clause is used to filter records based on specified conditions. It can be applied to SELECT, UPDATE, and DELETE statements.

a) Basic Syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

b) Example (Simple WHERE Clause):

```
SELECT * FROM employees
```

```
WHERE department_id = 10;
```

This selects all records from the employees table where the department_id is 10.

c) Operators in WHERE Clause:

- **=**: Equal to
- **!=** or **<>**: Not equal to
- **>**, **<**, **>=**, **<=**: Comparison operators
- **LIKE**: Pattern matching
- **BETWEEN**: Range filtering
- **IN**: Match any value in a list

d) Example (WHERE with Comparison Operator):

```
SELECT first_name, last_name FROM employees
```

```
WHERE salary > 50000;
```

This selects all employees with a salary greater than 50,000.

e) Example (WHERE with LIKE Operator):

```
SELECT * FROM employees
```

```
WHERE first_name LIKE 'J%';
```

This selects all employees whose first name starts with the letter "J".

Compound WHERE Clause with Multiple AND & OR Conditions

The **compound WHERE clause** allows you to filter records based on multiple conditions using logical operators AND and OR.

a) AND Operator:

- Used to combine multiple conditions where **all** conditions must be true for the record to be selected.

```
/* Print Details Of The Workers Whose FirstName  
Ends With 't' */  
SELECT *  
FROM Worker  
WHERE FirstName LIKE '%t';
```

```
/* Print Details Of The Workers Whose FirstName  
Contains 't' */  
SELECT *  
FROM Worker  
WHERE FirstName LIKE '%t%';
```


- **Example:**

```
SELECT * FROM employees
```

```
WHERE department_id = 10 AND salary > 60000;
```

This selects all employees who work in department 10 and have a salary greater than 60,000.

b) OR Operator:

- Used to combine multiple conditions where **at least one** condition must be true for the record to be selected.

- **Example:**

```
SELECT * FROM employees
```

```
WHERE department_id = 10 OR department_id = 20;
```

This selects all employees who work in either department 10 or department 20.

c) Combining AND and OR:

You can combine AND and OR in a compound WHERE clause. Be mindful of parentheses to ensure the correct logic is applied.

- **Example:**

```
SELECT * FROM employees
```

```
WHERE (department_id = 10 OR department_id = 20)
```

```
AND salary > 50000;
```

This selects employees who work in either department 10 or 20 **and** have a salary greater than 50,000.

Joins in SQL

Joins are used to retrieve data from multiple tables based on related columns between them. There are several types of joins:

a) Inner Join:

- An **Inner Join** returns only the rows that have matching values in both tables.
- **Syntax:**

```
SELECT columns
```

FROM table1

INNER JOIN table2

ON table1.column = table2.column;

- **Example:**

```
SELECT employees.first_name, departments.department_name
```

```
FROM employees
```

```
INNER JOIN departments
```

```
ON employees.department_id = departments.department_id;
```

This selects the first name of employees and their corresponding department name where the department_id matches in both tables.

b) Left Join (or Left Outer Join):

- A **Left Join** returns all rows from the left table, and the matching rows from the right table. If there is no match, NULL values are returned for columns from the right table.
- **Syntax:**

```
SELECT columns
```

```
FROM table1
```

```
LEFT JOIN table2
```

```
ON table1.column = table2.column;
```

- **Example:**

```
SELECT employees.first_name, departments.department_name
```

```
FROM employees
```

```
LEFT JOIN departments
```

```
ON employees.department_id = departments.department_id;
```

This returns all employees, even if they don't belong to a department. For employees without a department, department_name will be NULL.

c) Right Join (or Right Outer Join):

- A **Right Join** returns all rows from the right table, and the matching rows from the left table. If there is no match, NULL values are returned for columns from the left table.
- **Example:**

```
SELECT employees.first_name, departments.department_name
```

```
FROM employees
```

```
RIGHT JOIN departments
```

```
ON employees.department_id = departments.department_id;
```

This selects all departments and their employees, even if some departments don't have employees. The employee columns will contain NULL if there are no matches.

d) Full Join (or Full Outer Join):

- A **Full Join** returns all rows from both tables, with NULLs in places where there is no match.
- **Syntax:**

```
SELECT columns
```

```
FROM table1
```

```
FULL OUTER JOIN table2
```

```
ON table1.column = table2.column;
```

- **Example:**

```
SELECT employees.first_name, departments.department_name
```

```
FROM employees
```

```
FULL OUTER JOIN departments
```

```
ON employees.department_id = departments.department_id;
```

This selects all employees and departments, and displays NULL where there is no match.

Summary

- **INSERT**, **UPDATE**, and **DELETE** are key DML commands to manipulate data in a table.
- The **WHERE** clause filters records based on specific conditions.
- **Compound WHERE** clauses can combine multiple conditions using AND and OR.
- **Joins** are used to retrieve data from multiple related tables, with common types being Inner Join, Left Join, Right Join, and Full Join.

Sub-queries- Simple & Correlated Using IN, EXISTS, NOT EXISTS

Sub-queries in SQL

A **sub-query** (or nested query) is a query within another SQL query. Sub-queries can be used to perform operations that require multiple steps or to return data that will be used in the main query. They can be classified into two main types: **Simple Sub-queries** and **Correlated Sub-queries**.

1. Simple Sub-queries

A **simple sub-query** is a standalone query that can be executed independently of the main query. It typically returns a single value or a set of values that can be used in the main query.

Using IN Operator

The IN operator allows you to specify multiple values in a WHERE clause. You can use a simple sub-query with the IN operator to filter records based on the results of the sub-query.

- **Syntax:**

```
SELECT column1, column2
```

```
FROM table1
```

```
WHERE column_name IN (SELECT column_name FROM table2 WHERE condition);
```

- **Example:**

```
SELECT first_name, last_name
```

```
FROM employees
```

```
WHERE department_id IN (SELECT department_id FROM departments WHERE department_name = 'Sales');
```

In this example, the sub-query retrieves department_id values from the departments table where department_name is 'Sales', and the main query selects employees whose department_id matches those values.

Using EXISTS Operator

The EXISTS operator is used to test for the existence of any record in a sub-query. It returns TRUE if the sub-query returns one or more records, and FALSE if it returns no records.

- **Syntax:**

```
SELECT column1, column2
```

FROM table1

WHERE EXISTS (SELECT column_name FROM table2 WHERE condition);

- **Example:**

```
SELECT first_name, last_name
```

```
FROM employees
```

```
WHERE EXISTS (SELECT * FROM departments WHERE departments.department_id =  
employees.department_id);
```

Here, the main query selects employees for whom there is a matching record in the departments table based on department_id.

Using NOT EXISTS Operator

The NOT EXISTS operator is the opposite of EXISTS. It returns TRUE if the sub-query returns no records.

- **Syntax:**

```
SELECT column1, column2
```

```
FROM table1
```

```
WHERE NOT EXISTS (SELECT column_name FROM table2 WHERE condition);
```

- **Example:**

```
SELECT first_name, last_name
```

```
FROM employees
```

```
WHERE NOT EXISTS (SELECT * FROM departments WHERE departments.department_id =  
employees.department_id);
```

This example retrieves employees who do not belong to any department listed in the departments table.

2. Correlated Sub-queries

A **correlated sub-query** is a sub-query that refers to columns from the outer query. This means that the sub-query cannot be executed independently of the outer query because it relies on the outer query for its values.

Using IN Operator with Correlated Sub-query

- **Syntax:**

```
SELECT column1, column2
```

```
FROM table1 t1
```

```
WHERE column_name IN (SELECT column_name FROM table2 t2 WHERE t1.column_name =  
t2.column_name);
```

- **Example:**

```
SELECT e.first_name, e.last_name
```

```
FROM employees e
```

```
WHERE e.department_id IN (SELECT d.department_id FROM departments d WHERE d.manager_id =  
e.manager_id);
```

In this example, the inner query filters departments based on the manager_id from the outer employees table.

Using EXISTS with Correlated Sub-query

- **Syntax:**

```
SELECT column1, column2
```

```
FROM table1 t1
```

```
WHERE EXISTS (SELECT column_name FROM table2 t2 WHERE t1.column_name = t2.column_name);
```

- **Example:**

```
SELECT e.first_name, e.last_name
```

```
FROM employees e
```

```
WHERE EXISTS (SELECT * FROM departments d WHERE d.department_id = e.department_id AND  
d.location = 'New York');
```

This retrieves employees who are part of departments located in New York.

Using NOT EXISTS with Correlated Sub-query

- **Syntax:**

```
SELECT column1, column2
```

```
FROM table1 t1
```

```
WHERE NOT EXISTS (SELECT column_name FROM table2 t2 WHERE t1.column_name =  
t2.column_name);
```

- **Example:**

```
SELECT e.first_name, e.last_name
```

```
FROM employees e
```

```
WHERE NOT EXISTS (SELECT * FROM departments d WHERE d.department_id = e.department_id  
AND d.manager_id IS NULL);
```

This query retrieves employees whose departments have no assigned manager.

Summary of Sub-queries

Type	Description	Example
Simple Sub-query	A sub-query that can be executed independently and returns a single value or set of values.	SELECT first_name FROM employees WHERE department_id IN (SELECT department_id FROM departments);
Correlated Sub-query	A sub-query that depends on the outer query and cannot be executed independently.	SELECT e.first_name FROM employees e WHERE EXISTS (SELECT * FROM departments d WHERE d.manager_id = e.manager_id);
IN	Checks if a value is present in the result set of the sub-query.	WHERE department_id IN (SELECT department_id FROM departments WHERE department_name = 'Sales');
EXISTS	Checks for the existence of rows returned by the sub-query.	WHERE EXISTS (SELECT * FROM departments WHERE departments.department_id = employees.department_id);
NOT EXISTS	Checks for the absence of rows returned by the sub-query.	WHERE NOT EXISTS (SELECT * FROM departments WHERE departments.department_id = employees.department_id);

DCL statement Grant, Revoke, Group by clause, having clause

DCL Statements: GRANT and REVOKE

DCL (Data Control Language) statements are used to control access to data in a database. The primary DCL commands are GRANT and REVOKE, which manage permissions and access rights.

1. GRANT Statement

The GRANT statement is used to give users access privileges to database objects such as tables, views, and procedures.

Syntax:

```
GRANT privilege_type
```

```
ON object_name
```

```
TO user_name;
```

- **Example:**

```
GRANT SELECT, INSERT
```

```
ON employees
```

```
TO user1;
```

This example grants the SELECT and INSERT privileges on the employees table to user1.

Granting All Privileges:

You can grant all privileges on a table to a user.

```
GRANT ALL PRIVILEGES
```

```
ON employees
```

```
TO user1;
```

2. REVOKE Statement

The REVOKE statement is used to remove previously granted privileges from users.

Syntax:

```
REVOKE privilege_type
```

```
ON object_name
```

```
FROM user_name;
```

- **Example:**

```
REVOKE INSERT
```

```
ON employees
```


FROM user1;

This example removes the INSERT privilege on the employees table from user1.

Revoking All Privileges:

You can revoke all privileges from a user.

```
REVOKE ALL PRIVILEGES
```

```
ON employees
```

```
FROM user1;
```

GROUP BY Clause

The GROUP BY clause is used in collaboration with aggregate functions (like COUNT, SUM, AVG, etc.) to group the result set by one or more columns.

Syntax:

```
SELECT column1, aggregate_function(column2)
```

```
FROM table_name
```

```
WHERE condition
```

```
GROUP BY column1;
```

Example:

```
SELECT department_id, COUNT(*)
```

```
FROM employees
```

```
GROUP BY department_id;
```

This query counts the number of employees in each department, grouping the results by department_id.

Multiple Columns:

You can group by multiple columns.

```
SELECT department_id, job_id, COUNT(*)
```

```
FROM employees
```

```
GROUP BY department_id, job_id;
```

This query counts employees grouped by both department_id and job_id.

HAVING Clause

The HAVING clause is used to filter records that work on summarized group results, which is often used in conjunction with the GROUP BY clause. It is similar to the WHERE clause, but HAVING is applied after grouping.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

Example:

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;
```

This query returns departments with more than 10 employees.

Using HAVING with Multiple Conditions:

You can use logical operators in the HAVING clause to apply multiple conditions.

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 60000 AND COUNT(*) > 5;
```

This query retrieves departments with an average salary greater than 60,000 and more than 5 employees.

Summary of DCL and Grouping Clauses

Command	Description	Example
GRANT	Gives privileges to users for database objects.	GRANT SELECT, INSERT ON employees TO user1;

Command	Description	Example
REVOKE	Removes previously granted privileges from users.	REVOKE INSERT ON employees FROM user1;
GROUP BY	Groups rows that have the same values in specified columns into summary rows.	SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;
HAVING	Filters results after aggregation, working on grouped records.	SELECT department_id, COUNT(*) FROM employees GROUP BY department_id HAVING COUNT(*) > 10;

Views, Benefits of views, creating views, alter views and drop views

Views in SQL

A **view** is a virtual table in a database that is based on the result set of a SQL query. It does not store the data itself but provides a way to represent data from one or more tables in a specific format.

1. Creating Views

You can create a view using the CREATE VIEW statement. This statement allows you to define a view based on a SELECT query.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW employee_view AS
SELECT first_name, last_name, department_id
FROM employees
WHERE status = 'active';
```

In this example, employee_view is a view that includes only active employees from the employees table.

2. Benefits of Views

- **Data Abstraction:** Views provide a way to present data to users in a simplified manner without exposing the underlying table structure.
- **Security:** You can grant access to views rather than the underlying tables, limiting users' access to specific data.
- **Simplified Queries:** Views can encapsulate complex queries, making it easier for users to retrieve data without needing to know the underlying query structure.
- **Consistency:** Views ensure that users see a consistent dataset, even if the underlying tables change.
- **Join and Aggregate Data:** Views can combine data from multiple tables, allowing for more complex reporting and analysis without changing the underlying data structure.

3. Altering Views

You can modify an existing view using the CREATE OR REPLACE VIEW statement. This allows you to redefine the view with a new query.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS
SELECT new_column1, new_column2, ...
FROM new_table_name
WHERE new_condition;
```

Example:

```
CREATE OR REPLACE VIEW employee_view AS
SELECT first_name, last_name, department_id, hire_date
FROM employees
WHERE status = 'active' AND hire_date >= '2020-01-01';
```

This example updates the employee_view to include the hire_date column and filters for employees hired after January 1, 2020.

4. Dropping Views

You can remove a view from the database using the DROP VIEW statement. This permanently deletes the view definition.

Syntax:

```
DROP VIEW view_name;
```

Example:

DROP VIEW employee_view;

In this example, the employee_view is dropped from the database, and it will no longer be available for use.

Summary of Views

Aspect	Description	Example
Creating Views	Defines a virtual table based on a SELECT query.	CREATE VIEW employee_view AS SELECT first_name, last_name FROM employees;
Benefits of Views	- Data abstraction - Security - Simplified queries - Consistency - Join and aggregate data	N/A
Altering Views	Modifies the definition of an existing view.	CREATE OR REPLACE VIEW employee_view AS SELECT * FROM employees WHERE status = 'active';
Dropping Views	Removes a view from the database.	DROP VIEW employee_view;

Joins (inner join, outer join, cross join, self join), write complex queries using joins

Joins in SQL

Joins are used in SQL to combine rows from two or more tables based on a related column between them. There are several types of joins, including **INNER JOIN**, **OUTER JOIN**, **CROSS JOIN**, and **SELF JOIN**.

1. INNER JOIN

The **INNER JOIN** keyword selects records that have matching values in both tables.

Syntax:

SELECT columns

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

Example:

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
INNER JOIN departments d ON e.department_id = d.department_id;
```

This query retrieves employee names along with their department names for those employees who belong to a department.

2. OUTER JOIN

OUTER JOIN can be classified into three types: **LEFT JOIN**, **RIGHT JOIN**, and **FULL OUTER JOIN**.

- **LEFT JOIN:** Returns all records from the left table and matched records from the right table; if no match, NULL values are returned for right table columns.

Syntax:

```
SELECT columns  
FROM table1  
LEFT JOIN table2  
ON table1.common_column = table2.common_column;
```

Example:

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
LEFT JOIN departments d ON e.department_id = d.department_id;
```

This query retrieves all employees and their department names. If an employee does not belong to any department, the department_name will be NULL.

- **RIGHT JOIN:** Returns all records from the right table and matched records from the left table; if no match, NULL values are returned for left table columns.

Syntax:

```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.common_column = table2.common_column;
```

Example:

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
RIGHT JOIN departments d ON e.department_id = d.department_id;
```

This query retrieves all departments and the employees working in them. If a department has no employees, the employee columns will be NULL.

- **FULL OUTER JOIN:** Returns all records when there is a match in either left or right table records.

Syntax:

```
SELECT columns  
FROM table1  
FULL OUTER JOIN table2  
ON table1.common_column = table2.common_column;
```

Example:

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
FULL OUTER JOIN departments d ON e.department_id = d.department_id;
```

This query retrieves all employees and departments, including employees without departments and departments without employees.

3. CROSS JOIN

The **CROSS JOIN** returns the Cartesian product of two tables, meaning it will return all possible combinations of rows from both tables.

Syntax:

```
SELECT columns  
FROM table1  
CROSS JOIN table2;
```

Example:

```
SELECT e.first_name, d.department_name
```

```
FROM employees e
```

```
CROSS JOIN departments d;
```

This query retrieves every employee's name combined with every department name.

4. SELF JOIN

A **SELF JOIN** is a regular join but the table is joined with itself. It is often used to compare rows within the same table.

Syntax:

```
SELECT a.columns, b.columns
```

```
FROM table a, table b
```

```
WHERE condition;
```

Example:

```
SELECT e1.first_name AS Employee, e2.first_name AS Manager
```

```
FROM employees e1
```

```
INNER JOIN employees e2 ON e1.manager_id = e2.employee_id;
```

This query retrieves employee names along with their managers' names from the same employees table.

Complex Queries Using Joins

Here are a few complex queries that utilize different types of joins:

Example 1: Joining Multiple Tables

```
SELECT e.first_name, e.last_name, d.department_name, p.project_name
```

```
FROM employees e
```

```
INNER JOIN departments d ON e.department_id = d.department_id
```

```
INNER JOIN projects p ON e.employee_id = p.employee_id
```

```
WHERE d.location = 'New York';
```

This query retrieves employees from departments located in New York and the projects they are working on.

Example 2: Combining LEFT JOIN and GROUP BY


```
SELECT d.department_name, COUNT(e.employee_id) AS Employee_Count
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

This query counts the number of employees in each department, including departments that have no employees.

Example 3: Using HAVING with Joins

```
SELECT d.department_name, AVG(e.salary) AS Average_Salary
FROM departments d
INNER JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING AVG(e.salary) > 50000;
```

This query retrieves departments with an average salary greater than 50,000.

Example 4: CROSS JOIN with Filtering

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
CROSS JOIN departments d
WHERE d.location = 'California' AND e.department_id = d.department_id;
```

This query retrieves employee names and department names for those departments located in California, showcasing a combination of CROSS JOIN and filtering.

Summary of Joins

Type of Join	Description	Example
INNER JOIN	Returns rows with matching values in both tables.	SELECT e.first_name, d.department_name FROM employees e INNER JOIN departments d ON e.department_id = d.department_id;
LEFT JOIN	Returns all rows from the left table and matched rows from the right table; NULLs if no match.	SELECT e.first_name, d.department_name FROM employees e LEFT JOIN departments d ON e.department_id = d.department_id;

Type of Join	Description	Example
RIGHT JOIN	Returns all rows from the right table and matched rows from the left table; NULLs if no match.	SELECT e.first_name, d.department_name FROM employees e RIGHT JOIN departments d ON e.department_id = d.department_id;
FULL OUTER JOIN	Returns all rows when there is a match in either left or right table records.	SELECT e.first_name, d.department_name FROM employees e FULL OUTER JOIN departments d ON e.department_id = d.department_id;
CROSS JOIN	Returns the Cartesian product of two tables.	SELECT e.first_name, d.department_name FROM employees e CROSS JOIN departments d;
SELF JOIN	Joins a table to itself to compare rows.	SELECT e1.first_name AS Employee, e2.first_name AS Manager FROM employees e1 INNER JOIN employees e2 ON e1.manager_id = e2.employee_id;

Introduction to Store programs in MySql, Advantages of Stored program

Introduction to Stored Programs in MySQL

Stored Programs in MySQL are routines that are stored in the database and can be executed by calling them. They help encapsulate complex operations and logic within the database, allowing for more efficient and maintainable code. Stored programs include:

1. **Stored Procedures:** These are collections of SQL statements that can be executed as a single unit. They can accept parameters, perform operations, and return results.
2. **Stored Functions:** These are similar to stored procedures but are used to compute and return a single value.
3. **Triggers:** These are special types of stored programs that automatically execute in response to certain events on a specified table, such as INSERT, UPDATE, or DELETE.
4. **Events:** These are scheduled tasks that execute at specified intervals.

Advantages of Stored Programs

Stored programs provide several benefits that enhance database management and application development:

1. **Improved Performance:**
 - **Reduced Network Traffic:** Since the logic is stored in the database, multiple SQL statements can be executed with a single call, minimizing the amount of data sent over the network.
 - **Execution Plan Reuse:** Stored programs allow the database to reuse execution plans, which can speed up query processing.

2. Enhanced Security:

- **Controlled Access:** Permissions can be granted to execute stored programs without giving users direct access to underlying tables, providing an additional layer of security.
- **Data Validation:** Logic can be centralized within stored programs, ensuring data integrity and validation before operations are performed.

3. Modularity and Reusability:

- **Code Reusability:** Common logic can be written once in a stored program and reused across multiple applications, reducing code duplication and maintenance efforts.
- **Easier Updates:** Changes can be made to the stored program without altering the application code, simplifying updates and maintenance.

4. Business Logic Encapsulation:

- **Centralized Business Logic:** Business rules can be enforced at the database level, ensuring consistency across different applications that access the database.

5. Error Handling:

- **Built-in Error Management:** Stored programs can include error handling mechanisms, allowing developers to manage exceptions more effectively.

6. Complex Operations:

- **Support for Complex Logic:** Stored procedures can include control flow structures (such as loops and conditional statements), enabling complex business logic to be implemented directly in the database.

7. Scheduled Tasks:

- **Automation:** Stored programs, especially events, can automate routine tasks (like data archiving or cleanup) at specified intervals without manual intervention.

Example of a Stored Procedure

Here's a simple example of creating a stored procedure in MySQL:

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeeCountByDepartment(IN dept_id INT)
```

```
BEGIN
```

```
    SELECT COUNT(*) AS EmployeeCount
```

```
    FROM employees
```

```
    WHERE department_id = dept_id;
```

END //

DELIMITER ;

This stored procedure GetEmployeeCountByDepartment accepts a department ID as input and returns the count of employees in that department.

Implementing Constraints (primary key, foreign Key, unique, not null, default, check)

Implementing Constraints in SQL

In SQL, **constraints** are rules applied to columns in a table to ensure the integrity, accuracy, and consistency of data. Constraints are critical to maintain the quality of the data stored in the database. They can be applied when creating or altering a table.

The main types of constraints are:

- **Primary Key**
- **Foreign Key**
- **Unique**
- **Not Null**
- **Default**
- **Check**

1. Primary Key Constraint

A **Primary Key** is a column (or a combination of columns) that uniquely identifies each row in a table. Each table can have only one primary key, and it ensures that:

- The column(s) must contain **unique** values.
- The column(s) cannot contain **NULL** values.
- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype PRIMARY KEY,  
    column_name2 datatype,  
    ...
```

);

- **Example:**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50)  
);
```

This creates an employees table where employee_id is the primary key, meaning each employee must have a unique employee_id, and it cannot be NULL.

- **Composite Primary Key:** A primary key can be a combination of more than one column, known as a composite key.

```
CREATE TABLE orders (  
    order_id INT,  
    product_id INT,  
    PRIMARY KEY (order_id, product_id)  
);
```

2. Foreign Key Constraint

A **Foreign Key** is a column or set of columns in a table that establishes a link between data in two tables. It enforces referential integrity by ensuring that values in the foreign key column must match values in the primary key column of another table.

- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype,  
    column_name2 datatype,  
    FOREIGN KEY (column_name) REFERENCES parent_table(parent_column)  
);
```

- **Example:**

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

Here, department_id in the employees table is a foreign key that references the department_id in the departments table. This ensures that department_id in employees must have a corresponding value in the departments table.

3. Unique Constraint

A **Unique** constraint ensures that all values in a column (or combination of columns) are distinct. It allows **NULL** values, unlike the primary key constraint, but still ensures uniqueness.

- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype UNIQUE  
);
```

- **Example:**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE,  
    first_name VARCHAR(50)  
);
```

In this example, the email column must contain unique values for each employee, but it can contain NULL values if necessary.

- **Composite Unique:** A unique constraint can also be applied to a combination of columns.

```
CREATE TABLE orders (  
    order_id INT,  
    product_id INT,  
    UNIQUE (order_id, product_id)  
);
```

This ensures that the combination of order_id and product_id must be unique.

4. Not Null Constraint

A **Not Null** constraint ensures that a column cannot contain a **NULL** value. By default, columns in a table can contain NULL unless this constraint is applied.

- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype NOT NULL  
);
```

- **Example:**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL  
);
```

In this example, the first_name and last_name columns must contain values and cannot be left NULL.

5. Default Constraint

A **Default** constraint provides a default value for a column if no value is provided when inserting data. This helps to ensure that every column has a value, even if the user doesn't explicitly provide one.

- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype DEFAULT value
```

```
column_name datatype DEFAULT default_value  
);
```

- **Example:**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    hire_date DATE DEFAULT CURRENT_DATE  
);
```

In this case, if no hire_date is provided when a new employee is inserted, the default value will be the current date.

6. Check Constraint

A **Check** constraint ensures that all values in a column satisfy a specific condition. This allows you to enforce additional rules on the data that can be inserted.

- **Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype CHECK (condition)  
);
```

- **Example:**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    salary DECIMAL(10, 2) CHECK (salary > 0)  
);
```

In this example, the salary column must have a value greater than 0. Any attempt to insert or update a salary with a value less than or equal to 0 will result in an error.

Summary of SQL Constraints

Constraint	Description
Primary Key	Uniquely identifies each row in a table and ensures no NULL values.
Foreign Key	Ensures referential integrity between two tables by linking one column to another table's primary key.
Unique	Ensures all values in a column or combination of columns are unique, allowing NULL values.
Not Null	Ensures a column cannot have a NULL value.
Default	Provides a default value for a column when no value is specified.
Check	Ensures that values in a column meet a specific condition.

Introduction to cursors, types of cursors, advantages and disadvantages of cursors.

Introduction to Cursors

A cursor is a database object used to retrieve, manipulate, and navigate through the result set (the collection of rows) retrieved from a query. Cursors allow row-by-row processing of the data, giving developers more control over how records are accessed and modified. Unlike simple SQL queries that process all rows simultaneously, cursors provide mechanisms to traverse through each record one by one.

Types of Cursors

1. Implicit Cursor:

- Automatically created by the database system for single row queries or SELECT statements that return only one row.

- They are managed internally, and developers typically don't need to define or open them explicitly.

2. Explicit Cursor:

- Defined explicitly by the developer to handle multiple rows returned by a query.
- Must be declared, opened, fetched, and closed manually.

Types of explicit cursors include:

- Static Cursor: The result set is determined when the cursor is opened and cannot be changed during its lifetime.
- Dynamic Cursor: Reflects changes made to the rows in the result set (like insertions or updates) while the cursor is open.
- Forward-only Cursor: Allows fetching rows in one direction only (from the first to the last row).
- Scroll Cursor: Allows moving both forward and backward through the result set.

3. Cursor for Loops:

- Simplifies the usage of explicit cursors by automatically opening, fetching, and closing the cursor during iteration.

4. Parameterised Cursor:

- Accepts parameters, allowing flexibility in fetching data based on varying inputs.

Advantages of Cursors

1. Row-by-row Processing:

- Cursors provide fine-grained control, allowing operations on individual rows. This is useful for complex business logic that requires iterative processing.

2. Handling Large Datasets:

- Instead of loading all data into memory, cursors handle records one at a time, making it feasible to work with large result sets.

3. Complex Processing:

- Ideal for scenarios where you need to perform complex operations on each row, which would be difficult to achieve with set-based operations.

4. Custom Navigation:

- Cursors allow moving forward or backward through a result set, skipping rows, or even fetching specific rows multiple times.

Disadvantages of Cursors

1. Performance Overhead:

- Cursors tend to be slower than set-based operations because they process data row by row, which can be inefficient for large datasets.

2. Resource-Intensive:

- Cursors consume more memory and CPU resources since they keep a lock on the result set and maintain pointers for navigation.

3. Concurrency Issues:

- Long-running cursors can lead to locking problems, where other transactions cannot access the same data until the cursor is closed.

4. Reduced Scalability:

- Due to the row-by-row processing nature, cursors do not scale well with large volumes of data, making them unsuitable for high-performance applications.

ACID Properties in DBMS

In the context of databases, ACID refers to a set of properties that ensure reliable processing of database transactions.

These properties guarantee that database operations are processed reliably and help to maintain the integrity of data.

The ACID properties are:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

1. Atomicity

- Definition: Atomicity ensures that each transaction is treated as a single, indivisible unit of work. A transaction will either complete fully (all operations are executed) or not at all (if any operation fails, the entire transaction fails). This ensures that partial updates never occur.
- Example:
 - Consider a banking system where you transfer \$100 from Account A to Account B. This transaction consists of two operations:
 1. Debit \$100 from Account A.
 2. Credit \$100 to Account B.

If the debit succeeds but the credit fails, atomicity ensures that the entire transaction is rolled back, and the money remains in Account A (the transfer is canceled). Without atomicity, the system could lose the \$100.

2. Consistency

- Definition: Consistency ensures that a transaction brings the database from one valid state to another valid state, maintaining all predefined rules, such as integrity constraints (e.g., foreign key, unique key). The database must remain in a consistent state before and after the transaction.
- Example:
 - Consider an e-commerce application where the total quantity of a product is tracked. If you place an order to purchase 5 units of an item and the

stock has 10 units, after the transaction the stock should be updated to 5. The consistency property ensures that the stock never goes below zero (e.g., ordering more than the available quantity).

3. Isolation

- Definition: Isolation ensures that the operations of one transaction are invisible to other concurrent transactions until the transaction is completed. This prevents conflicting operations from occurring when multiple transactions run simultaneously.
- Example:
 - Imagine two users trying to buy the last available unit of a product at the same time. Isolation ensures that only one of the transactions will succeed in purchasing the product, and the other transaction will see the updated state after the first transaction has completed. This prevents both users from purchasing the same item.

There are different isolation levels, such as:

- Read Uncommitted: A transaction can read data from another uncommitted transaction (can lead to dirty reads).
- Read Committed: A transaction cannot read data from another uncommitted transaction.
- Repeatable Read: Ensures the same result is returned on multiple reads within a transaction.

- Serializable: Complete isolation; transactions are executed sequentially, which prevents conflicts.

4. Durability

- Definition: Durability guarantees that once a transaction is committed, its changes are permanent. Even in the case of a system crash or failure, the committed transaction's changes will not be lost.
- Example:
 - After a user successfully books a flight ticket, even if the server crashes right after the transaction completes, the booking will still be reflected in the system when it comes back online. This is because the changes have been made permanent in the database.

Example:

Let's consider a banking transaction as an example of ACID properties in action:

Transaction: Transfer \$500 from Account X to Account Y.

- Atomicity: The transaction should either debit \$500 from Account X and credit \$500 to Account Y, or neither operation should happen. If there's an error, the transaction is rolled back.
- Consistency: The total balance of both accounts before and after the transaction should remain unchanged. If Account X has \$1,000 and Account Y has \$2,000, after

the transaction, Account X should have \$500, and Account Y should have \$2,500.

- Isolation: If two users are transferring money simultaneously, one transaction should not affect the outcome of the other. For example, if User 1 transfers \$500 from Account X to Y, User 2 should not see an intermediate state where only \$500 has been deducted from X but not yet added to Y.
- Durability: Once the transaction is committed, it should be saved in the database, even in the case of a system crash or power failure. After the transaction is completed, the new balances of both accounts should be retained in the database.

Summary of ACID Properties:

1. Atomicity: All or nothing, the transaction is fully completed or aborted.
2. Consistency: Database remains in a valid state before and after the transaction.
3. Isolation: Transactions are independent of each other until committed.
4. Durability: Once committed, changes made by the transaction are permanent.

Introduction to Triggers, types of triggers, Syntax For Creating Triggers.

Types of Triggers in SQL

Following are the six types of triggers in SQL:

1. AFTER INSERT Trigger

This trigger is invoked after the insertion of data in the table.

2. AFTER UPDATE Trigger

This trigger is invoked in SQL after the modification of the data in the table.

3. AFTER DELETE Trigger

This trigger is invoked after deleting the data from the table.

4. BEFORE INSERT Trigger

This trigger is invoked before the inserting the record in the table.

5. BEFORE UPDATE Trigger

This trigger is invoked before the updating the record in the table.

6. BEFORE DELETE Trigger

This trigger is invoked before deleting the record from the table.

Syntax of Trigger in SQL

1. CREATE TRIGGER Trigger_Name
2. [BEFORE | AFTER] [Insert | Update | Delete]
3. ON [Table_Name]
4. [FOR EACH ROW | FOR EACH COLUMN]
5. AS
6. Set of SQL Statement

Types of Triggers in SQL

Triggers in SQL are special procedures that are automatically executed or fired when a specific event occurs in a database. These events could be INSERT, UPDATE, or DELETE

operations. There are six main types of triggers, categorized based on the timing (before or after the event) and the event itself (INSERT, UPDATE, DELETE).

1. AFTER INSERT Trigger

- Description: This trigger is invoked after a record is inserted into a table. It is often used to log the insertion or take action after a new row is added.
- Use Case: Logging newly inserted rows, updating related tables, or performing additional calculations.
- Example:

```
CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log (emp_id, name, action,
    action_time)
    VALUES (NEW.emp_id, NEW.name, 'Inserted', NOW());
END;
```

- Explanation:
 - Whenever a new row is inserted into the employees table, the trigger logs the action in the

employee_log table, capturing the employee ID, name, and the timestamp.

2. AFTER UPDATE Trigger

- Description: This trigger is executed after a record is updated in the table. It is useful for tracking changes or performing further updates to related data.
- Use Case: Tracking historical changes to data or triggering additional business logic after an update.
- Example:

```
CREATE TRIGGER after_employee_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log (emp_id, old_salary,
new_salary, action_time)
    VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
END;
```

- Explanation:
 - After any update to the employees table, this trigger logs the employee's old and new salary values, along with the timestamp, into the employee_log table.

3. AFTER DELETE Trigger

- Description: This trigger is fired after a record is deleted from the table. It can be used to archive deleted data or log the deletion event.
- Use Case: Logging deletions or cascading changes across other related tables.
- Example:

```
CREATE TRIGGER after_employee_delete
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_log (emp_id, name, action,
action_time)
    VALUES (OLD.emp_id, OLD.name, 'Deleted', NOW());
END;
```

- Explanation:
 - Whenever an employee is deleted from the employees table, this trigger logs the deletion event, capturing the employee ID, name, and the timestamp in the employee_log table.

4. BEFORE INSERT Trigger

- Description: This trigger is fired before a new record is inserted into the table. It is useful for validating or modifying data before it is inserted.
- Use Case: Data validation, setting default values, or ensuring certain conditions are met before insertion.
- Example:

```
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary
cannot be negative';
    END IF;
END;
```

- Explanation:
 - Before inserting a new employee into the employees table, this trigger checks if the salary is negative. If it is, an error is raised, and the insertion is prevented.

5. BEFORE UPDATE Trigger

- Description: This trigger is fired before an update operation is performed on a record. It can be used for data validation or automatically modifying values before the update.
- Use Case: Ensuring valid data updates or enforcing business rules before data changes.
- Example:

```
CREATE TRIGGER before_employee_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary
cannot be negative';
    END IF;
END;
```

- Explanation:
 - Before updating the employees table, this trigger ensures that the salary being updated is not negative. If it is, the update is blocked, and an error message is shown.

6. BEFORE DELETE Trigger

- Description: This trigger is fired before a record is deleted from the table. It is often used to validate conditions or restrict deletions.
- Use Case: Preventing the deletion of important records or performing checks before a delete operation.
- Example:

```
CREATE TRIGGER before_employee_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    IF OLD.emp_id = 1 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot
delete admin user';
    END IF;
END;
```

- Explanation:
 - Before deleting an employee from the employees table, this trigger checks if the employee has an ID of 1 (admin user). If so, the deletion is prevented, and an error is raised.

Summary of Trigger Types:

Trigger Type	Description	Typical Use Cases
AFTER INSERT	Fired after a row is inserted.	Logging new data, updating related tables.
AFTER UPDATE	Fired after a row is updated.	Logging changes, auditing, or maintaining related data.
AFTER DELETE	Fired after a row is deleted.	Logging deletions, archiving removed data.
BEFORE INSERT	Fired before a row is inserted.	Validating data before insertion, setting default values.
BEFORE UPDATE	Fired before a row is updated.	Validating data before updating, enforcing business rules.
BEFORE DELETE	Fired before a row is deleted.	Preventing or validating deletions, enforcing constraints.

Advantages of Using Triggers:

1. Automatic Execution:

- Triggers execute automatically in response to specified events, making them useful for enforcing rules or constraints without manual intervention.

2. Enforce Data Integrity:

- Triggers help maintain data integrity by ensuring that business rules and constraints are adhered to before or after data modification.

3. Consistency Across Tables:

- Triggers can ensure that related data in different tables remains consistent by automating updates or deletions in dependent tables.

4. Audit Trails:

- Triggers can log any changes to data, providing an audit trail that can help track who modified the data and when.

Disadvantages of Using Triggers:

1. Performance Overhead:

- Triggers can introduce performance issues, especially if they contain complex logic or operate on large datasets.

2. Hidden Logic:

- The logic inside triggers is hidden from the application layer, making debugging and troubleshooting more challenging.

3. Difficulty in Managing Multiple Triggers:

- When multiple triggers are defined on the same table or event, it can be difficult to manage and predict the order of execution.

4. Unintended Side Effects:

- Poorly designed triggers can lead to unintended consequences, such as infinite loops, if a trigger recursively calls another trigger.

Concurrent Transactions: Concurrency control, need for concurrency control, Locking Techniques

1. Concurrent Transactions

- **Definition:** Concurrent transactions refer to the execution of multiple transactions at the same time in a database system. Each transaction may involve read and write operations on the shared data.
- **Challenge:** When multiple transactions run simultaneously, there is a potential for conflicts like **lost updates, dirty reads, uncommitted data overwrites, and inconsistent data.**

2. Need for Concurrency Control

- **Concurrency Control:** It refers to the techniques and mechanisms used to ensure correct execution of

transactions in a concurrent environment while maintaining database consistency and integrity.

- **Problems without Concurrency Control:**

- **Lost Update:** Occurs when two transactions simultaneously read and update the same data, causing one update to be overwritten by another.
- **Dirty Read:** A transaction reads data that has been written by another uncommitted transaction, leading to incorrect results if the uncommitted transaction rolls back.
- **Unrepeatable Read:** A transaction reads the same data twice and gets different results because another transaction has modified the data between reads.
- **Phantom Reads:** A transaction retrieves a set of rows based on a condition, and later in the same transaction, re-executes the query and finds additional rows inserted by another transaction.

3. Locking Techniques

Locking is a widely used concurrency control mechanism that ensures that data used by one transaction is not simultaneously used by another in a way that could lead to inconsistencies.

a) Types of Locks:

- **Shared Lock (S-Lock):** Acquired by transactions that are only reading data. Multiple transactions can hold a

shared lock on the same data, allowing for concurrent reads.

- **Exclusive Lock (X-Lock):** Acquired by transactions that need to modify the data. Only one transaction can hold an exclusive lock at any time, preventing other transactions from accessing the data.

b) Lock Granularity:

- **Row-level Lock:** Locks only a specific row in a table, providing high concurrency but higher locking overhead.
- **Page-level Lock:** Locks a disk page, which could contain multiple rows. This provides a balance between concurrency and overhead.
- **Table-level Lock:** Locks the entire table, ensuring low overhead but limiting concurrency.

c) Two-Phase Locking Protocol (2PL):

- **Growing Phase:** A transaction can acquire locks but not release any.
- **Shrinking Phase:** Once the transaction starts releasing locks, it cannot acquire any more locks.
- **Purpose:** Guarantees **serializability** (ensuring that transactions execute as if they were in serial order), preventing lost updates and dirty reads.

d) Locking Schemes:

1. **Pessimistic Locking:** Assumes conflicts will occur and locks data before accessing it. Suitable for high-contention environments.
2. **Optimistic Locking:** Assumes conflicts are rare and does not lock data when reading but checks for conflicts before committing the transaction. Suitable for low-contention environments.

e) Deadlocks:

- **Definition:** A deadlock occurs when two or more transactions are waiting for each other to release locks, creating a cycle of dependencies that cannot be resolved.
- **Prevention Techniques:**
 - **Wait-Die Scheme:** Older transactions wait for younger ones to release locks, while younger ones are aborted.
 - **Wound-Wait Scheme:** Older transactions force younger ones to abort, while younger ones must wait for older ones.
 - **Timeouts:** Abort transactions that are waiting for too long.

4. Deadlock Detection and Recovery:

- **Detection:** Periodically checking for cycles in the wait-for graph (a graph where nodes represent transactions, and edges represent dependencies).

- **Recovery:** Once a deadlock is detected, one of the involved transactions is rolled back to break the cycle.

5. Other Concurrency Control Techniques:

- **Timestamp-based Protocols:** Transactions are ordered based on their timestamps to ensure consistency. Transactions with earlier timestamps get priority.
- **Validation-based Protocols:** Transactions proceed without locks but are validated at the end. If they pass validation, they are committed; otherwise, they are rolled back.

Database Recovery: Introduction, Need For Recovery, Types Of Errors Recovery Techniques

1. Introduction to Database Recovery

- **Definition:** Database recovery refers to the process of restoring a database to a consistent state after an unexpected failure or error. It ensures the durability and consistency of transactions, even in the face of crashes, errors, or data corruption.

- **Goal:** The primary goal of recovery is to ensure that committed transactions are preserved (durability) and that any uncommitted transactions are rolled back to maintain database integrity.

2. Need for Recovery

Databases need recovery mechanisms to ensure the following:

- **Data Integrity:** In the event of system failures, such as crashes or power outages, recovery ensures that the database is restored to a consistent state.
- **Durability (ACID Property):** The results of committed transactions must be preserved, even in case of failures.
- **Handling Transaction Failures:** When a transaction fails due to errors, the changes made by the transaction should be undone to prevent inconsistencies.
- **Prevent Data Loss:** Recovery mechanisms help avoid data loss in case of unexpected failures, ensuring minimal downtime and data restoration.

3. Types of Errors

Several types of errors can affect the database, making recovery mechanisms necessary:

a) Transaction Failure:

- **Reason:** Occurs when a transaction cannot complete successfully due to errors like invalid inputs, deadlock, or insufficient resources.
- **Impact:** Only the affected transaction needs to be rolled back or restarted.

b) System Crash:

- **Reason:** Caused by hardware failures, power outages, or operating system crashes. The system stops abruptly, potentially leaving transactions incomplete.
- **Impact:** Requires recovering from the last consistent state before the crash.

c) Media Failure:

- **Reason:** Occurs due to physical issues like hard disk crashes or data corruption. The storage media holding the database may become unavailable.
- **Impact:** Requires restoring the database from backups.

d) Logical Errors:

- **Reason:** Caused by software bugs, human errors (such as accidental deletion), or corruption of data structures within the database.
- **Impact:** Data corruption may spread if not identified early, requiring partial or full recovery of data.

4. Recovery Techniques

To handle various types of errors and failures, databases implement several recovery techniques:

a) Log-Based Recovery:

- **Concept:** Uses logs to keep a record of all the transactions that modify the database. The log contains information like transaction start, data changes, and commit status.
- **Steps:**
 1. **Write-Ahead Logging (WAL):** Before making changes to the database, the log is updated. This ensures that no change is applied before it is recorded.
 2. **Undo (Rollback):** If a transaction fails or is aborted, the database is rolled back to its original state using the log.
 3. **Redo (Roll Forward):** If a transaction was committed before a failure but changes weren't reflected in the database, the log helps apply these changes during recovery.

b) Checkpoints:

- **Concept:** A checkpoint is a saved point in time when the database state is considered consistent. All the transactions completed before the checkpoint are permanently saved in the database.
- **Benefits:**
 - Limits the amount of log information that needs to be processed during recovery.
 - Speeds up recovery by reducing the number of transactions to be redone or undone.

c) Deferred Update (No Immediate Write):

- **Concept:** Changes made by a transaction are not applied to the database until the transaction commits. If a transaction fails, no changes are written, simplifying recovery.
- **Advantages:** It eliminates the need to undo changes for failed transactions, as no changes are made until the commit point.

d) Immediate Update:

- **Concept:** The database is updated immediately as a transaction makes changes, but the changes are also recorded in the log. If a failure occurs, uncommitted changes are undone.
- **Steps:**
 1. Use logs to **undo** the changes made by incomplete transactions.
 2. Use logs to **redo** the changes for committed transactions that may not have been applied due to failure.

e) Shadow Paging:

- **Concept:** Shadow paging maintains two versions of the database: a current page table (in use) and a shadow page table (backup). The current page table is updated with new changes, but the shadow table remains unchanged.
- **Steps:**
 1. During updates, changes are made to a copy of the page (current page table), not the original data.
 2. If a transaction is committed, the new page table replaces the shadow page.

3. If a transaction fails, the system reverts to the shadow page table.
- **Advantages:** It provides fast recovery because there's no need for logging or undoing. The system only switches between the two page tables.

f) Backup and Restore:

- **Concept:** Regular backups are taken to prevent data loss in case of media failure or data corruption. When a failure occurs, the most recent backup is restored, and log files are applied to recover recent transactions.
- **Types of Backups:**
 1. **Full Backup:** A complete copy of the entire database.
 2. **Incremental Backup:** A copy of only the changes made since the last backup.
 3. **Differential Backup:** A copy of all changes made since the last full backup.

g) Recovery in Distributed Databases:

- **Concept:** In distributed databases, recovery is more complex due to multiple sites participating in a transaction.
- **2-Phase Commit (2PC):** A protocol used to ensure atomicity in distributed transactions. It involves:
 1. **Phase 1 (Prepare):** The coordinator asks all participating sites to prepare for commit.
 2. **Phase 2 (Commit):** If all sites are ready, the coordinator sends a commit signal; otherwise, it sends a rollback signal.